

# Souly

## Source code documentation for judge review

An AI study companion for children with learning differences. Souly teaches from the real Ministry of Education textbooks, gives hints instead of answers, and reports what a child actually did to both their teacher and their parents.

### COMPLETE SOURCE CODE

[github.com/beshoywael-him/souly-backend](https://github.com/beshoywael-him/souly-backend)

|             |                                                      |
|-------------|------------------------------------------------------|
| TEAM        | <b>Heliopolis Robocards</b>                          |
| TEAM NUMBER | <b>404</b>                                           |
| CATEGORY    | <b>Exhibition Senior</b>                             |
| SUBMISSION  | <b>Requirement 3 — source code for judge review</b>  |
| LANGUAGE    | <b>Python 3.12 · JavaScript · SQL</b>                |
| SIZE        | <b>87 files · 30,463 lines · 206 automated tests</b> |
| SUBMITTED   | <b>28 August 2026</b>                                |



Scan to open  
the repository

# What is in this document

---

Souly is 30,463 lines across 87 files. Printing all of it would be unreadable, so this document is a guided tour: the structure of the codebase, the technologies behind each layer, and thirteen extracts of the actual source chosen because each one shows a decision we made and why. The complete code is at the address on the cover.

|      |                                                      |                                       |
|------|------------------------------------------------------|---------------------------------------|
| 1    | <b>What Souly is, and how the parts fit together</b> | System overview and the three zones   |
| 2    | <b>How the files are organised</b>                   | Every folder, what it holds, and why  |
| 3    | <b>Tools and technologies</b>                        | What we chose, and what we rejected   |
| 4    | <b>Following one request through the system</b>      | A quiz answer, from tap to database   |
| 5    | <b>The code, with commentary</b>                     | Thirteen extracts from the repository |
| 5.1  | The database layer                                   | app/db.py                             |
| 5.2  | The flag lifecycle as data                           | app/models.py                         |
| 5.3  | Receiving a classroom detection                      | app/routers/flags.py                  |
| 5.4  | Refusing an illegal move                             | app/routers/flags.py                  |
| 5.5  | One writer for every number                          | app/economy.py                        |
| 5.6  | Finding the right page of the book                   | app/services/rag.py                   |
| 5.7  | Canon, rendition and path                            | app/services/tutor.py                 |
| 5.8  | Why Souly never gives the answer                     | app/services/llm.py                   |
| 5.9  | What happens when the network fails                  | app/services/llm.py                   |
| 5.10 | Security sized to the user                           | app/routers/auth.py · parent_auth.py  |
| 5.11 | The front end holds no truth                         | static/student/api.js                 |
| 5.12 | The test that protects the demo                      | tests/test_learning.py                |
| 6    | <b>Database design</b>                               | 41 tables, seven migrations           |
| 7    | <b>How we write and check our code</b>               | Conventions, comments, 206 tests      |
| 8    | <b>What runs today, and what is still ahead</b>      | An honest scope statement             |
| 9    | <b>Running the system yourself</b>                   | Setup, in order                       |

## How to read the code extracts

Every extract in section 5 is a screenshot of the real file, with the file path and the exact line numbers shown in the header bar. Nothing has been retyped, shortened or tidied for this document — if an extract shows lines 146 to 192 of `app/routers/flags.py`, those are the lines you will find at that path in the repository. Comments are shown as they are written in the source, because how the code is commented is part of what is being submitted.

### A NOTE ON WHAT IS NOT HERE

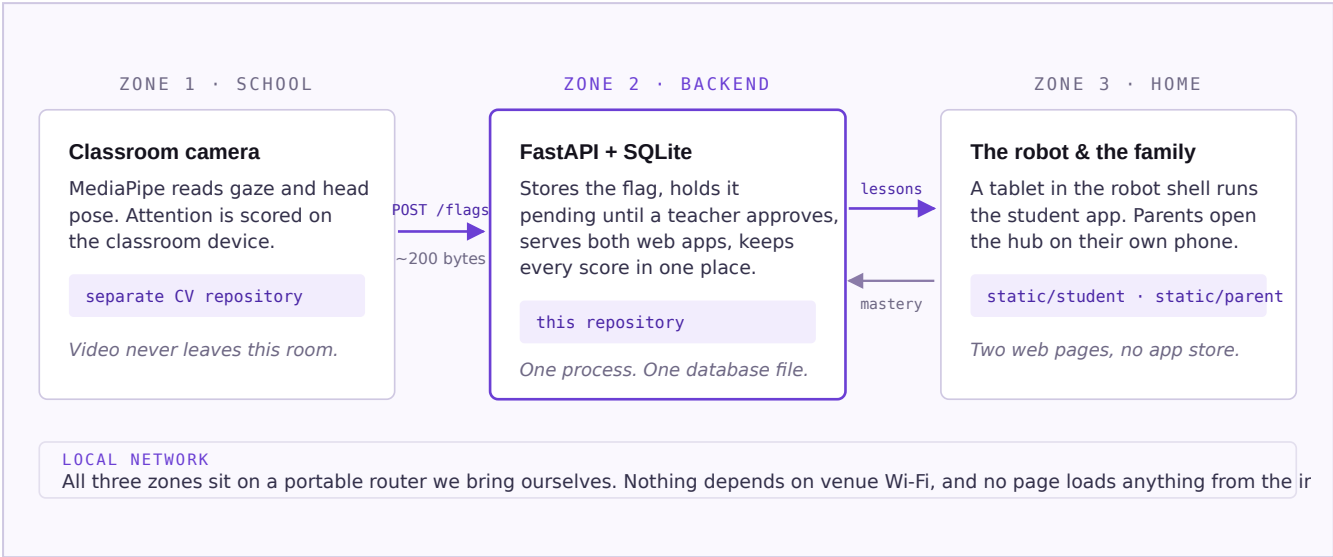
The classroom computer-vision code is owned by a separate team and lives in a separate repository. This document covers the backend, the two web front ends, the database and the tests. Section 1 shows where the vision system connects, and section 8 states plainly what is built and what is not.

# 1 What Souly is, and how the parts fit together

Souly closes a loop that no existing product closes: a child's difficulty is noticed at school, re-taught at home the same evening, and the result is visible to their teacher and their parents. Three zones, one database, one contract between them.

|                                                   |                                            |                                             |                                              |                                                                  |
|---------------------------------------------------|--------------------------------------------|---------------------------------------------|----------------------------------------------|------------------------------------------------------------------|
| <b>30,463</b><br>lines of code<br>across 87 files | <b>83</b><br>API operations<br>on 77 paths | <b>41</b><br>database tables<br>and 9 views | <b>206</b><br>automated tests<br>all passing | <b>106</b><br>real textbook<br>pages<br>from 2 Ministry<br>books |
|---------------------------------------------------|--------------------------------------------|---------------------------------------------|----------------------------------------------|------------------------------------------------------------------|

## 1.1 The three zones



**FIG 1** The system in three zones. The contract between zone 1 and zone 2 is deliberately tiny — a small JSON event, never an image — which is what lets us promise that video never leaves the classroom.

## 1.2 What each part is responsible for

| PART                      | RESPONSIBILITY                                                                                                   | WHERE IT LIVES             |
|---------------------------|------------------------------------------------------------------------------------------------------------------|----------------------------|
| <b>Classroom vision</b>   | Scores attention from the camera and publishes a small flag event when focus drifts past that child's threshold. | separate repository        |
| <b>Flag spine</b>         | Receives flags, filters noise, holds them pending, records every state change.                                   | app/routers/flags.py       |
| <b>Curriculum library</b> | Maps each lesson to its pages in the real Ministry PDFs and reads the page text at study time.                   | app/services/curriculum.py |
| <b>Tutor</b>              | Writes the explanation for this page for this child, and runs the four-step hint ladder.                         | app/services/tutor.py      |
| <b>Economy</b>            | The only code allowed to change a child's stars, XP, level or mastery.                                           | app/economy.py             |
| <b>Student app</b>        | Ten screens the child uses, drawn entirely from live API data.                                                   | static/student/            |
| <b>Parents' hub</b>       | Progress, teacher notes and messaging, scoped strictly to that parent's own children.                            | static/parent/             |

## 2 How the files are organised

---

The repository has one rule behind its shape: a file should be findable from the thing you want to change. If you want to change what a screen looks like you open a page file; if you want to change what a star is worth you open one constants block; if you want to add a table you add one migration file.

| FOLDER           | FILES | LINES  | WHAT IS IN IT                                                                                        |
|------------------|-------|--------|------------------------------------------------------------------------------------------------------|
| app/             | 28    | 10,627 | The backend. Config, database access, the economy, eleven routers and six services.                  |
| static/          | 13    | 8,304  | Both front ends — the student app and the parents' hub — as plain HTML, CSS and JavaScript.          |
| tests/           | 8     | 3,763  | 206 automated tests plus two scripts that drive the app in a real browser.                           |
| scripts/         | 16    | 3,178  | Setup and operations: database creation, seeding, textbook ingest, key checks, the demo walkthrough. |
| schema*.sql      | 7     | 2,184  | The database, as seven migrations applied in order.                                                  |
| docs/            | 6     | 1,823  | The stack decision record, the curriculum guide and the integration contract for the vision team.    |
| data/curriculum/ | 3     | 134    | The two Ministry PDFs and the human-approved map of lesson to page.                                  |
| (root)           | 6     | 450    | README, requirements, and the launch scripts for Windows, Linux and HTTPS.                           |

Counts exclude generated caches. The full per-file listing is section 10.

## 2.1 Inside app/

| FILE                    | LINES | PURPOSE                                                                                         |
|-------------------------|-------|-------------------------------------------------------------------------------------------------|
| main.py                 | 118   | Registers the eleven routers, serves both front ends, runs the migrations on start-up.          |
| config.py               | 125   | The only file in the project that reads environment variables.                                  |
| db.py                   | 134   | Opens SQLite with foreign keys and WAL on. Extract 5.1.                                         |
| models.py               | 215   | The request and response shapes for the flag API — the contract three teams share. Extract 5.2. |
| economy.py              | 557   | Stars, XP, levels, streaks, badges, mastery. Extract 5.5.                                       |
| security.py             | 58    | PBKDF2 hashing for parent access codes and picture passwords.                                   |
| deps.py                 | 47    | Resolves a student from the URL, or returns 404 — used by every student route.                  |
| onboarding_items.py     | 323   | The entry activity's items and their fixed prompt ladders.                                      |
| routers/learning.py     | 1,217 | Subjects, the lesson plan, lesson pages, practice generation, quizzes.                          |
| routers/parent.py       | 965   | The whole parents' hub API.                                                                     |
| routers/onboarding.py   | 498   | The one-time activity that learns how a child likes things explained.                           |
| routers/tutor_api.py    | 426   | Chat, hints, re-explain, and the one-request voice loop.                                        |
| routers/gamification.py | 423   | Games, the rewards shop, badges, the daily challenge.                                           |
| routers/flags.py        | 375   | The classroom spine. Extracts 5.3 and 5.4.                                                      |
| routers/auth.py         | 357   | The face picker and picture password. Extract 5.10.                                             |
| routers/student.py      | 316   | Profile, the home screen in one call, settings.                                                 |
| routers/parent_auth.py  | 303   | Parent sign-in, with its own separate token table.                                              |
| routers/progress.py     | 179   | The progress dashboard, derived entirely from the activity log.                                 |
| routers/health.py       | 49    | One call that reports whether keys are present and curriculum is loaded.                        |
| services/tutor.py       | 1,967 | Explanations, hints, question generation. Extract 5.7.                                          |
| services/llm.py         | 886   | The Gemini client and the offline fallback. Extracts 5.8 and 5.9.                               |

| FILE                   | LINES | PURPOSE                                                                            |
|------------------------|-------|------------------------------------------------------------------------------------|
| services/curriculum.py | 437   | Reads the real textbook pages from disk; renders and extracts them at ingest time. |
| services/tts.py        | 251   | Text to speech behind a swappable provider interface.                              |
| services/rag.py        | 250   | Finds the most relevant textbook page for a question. Extract 5.6.                 |
| services/stt.py        | 147   | Speech to text through ElevenLabs.                                                 |



## 3 Tools and technologies

---

Every choice below was made against the same two constraints: eight students had to be able to read each other's code, and the whole system had to run on a laptop and a portable router with no internet. Several times that meant choosing the less powerful option on purpose.

| LAYER                | WHAT WE USE                 | WHY THIS, AND WHAT WE REJECTED                                                                                                                                                                                                                                                                                |
|----------------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Language</b>      | Python 3.12                 | Not the newest release. MediaPipe, which the vision team needs, supports 3.9 to 3.12 only — so the whole team pins to the version the hardest dependency allows.                                                                                                                                              |
| <b>Web framework</b> | FastAPI + Uvicorn           | Chosen partly for <a href="#">/docs</a> : FastAPI generates a live, clickable API page from the code itself, so a teammate can test an endpoint without writing a client. That page is also how we hand the API to the interface squad.                                                                       |
| <b>Database</b>      | SQLite, raw SQL             | One file, no server to install or start, nothing to go wrong at a venue. <i>Rejected</i> : SQLAlchemy and every other ORM — an ORM would hide the SQL, and on a team learning databases this term the SQL is the part we want visible.                                                                        |
| <b>Front end</b>     | Plain HTML, CSS, JavaScript | <i>Rejected</i> : React and any build step. No <code>npm install</code> , no compilation — edit a file, refresh the browser. Any member can contribute on day one instead of spending a week on a toolchain.                                                                                                  |
| <b>Reasoning</b>     | Gemini 2.5 Pro              | Called over its plain REST API with <a href="#">httpx</a> . <i>Rejected</i> : the official SDK — one less package to install at a venue, and the request stays visible in our own file where we can debug it.                                                                                                 |
| <b>Retrieval</b>     | Our own TF-IDF search       | Roughly a hundred textbook pages is small. <i>Rejected</i> : ChromaDB with sentence-transformers — a vector database and a downloaded model would add setup and offline risk for no measurable gain at this size. The interface is written so we can swap the search without touching anything that calls it. |
| <b>Speech in</b>     | ElevenLabs Scribe           | The key stays on the server; the browser only sends audio. A key shipped to a tablet is a key on the open internet.                                                                                                                                                                                           |
| <b>Speech out</b>    | Provider-independent        | Three providers are implemented behind one interface and selected by a single environment variable, including the browser's own voice, which needs no network at all.                                                                                                                                         |
| <b>Content</b>       | Ministry PDFs + OCR         | The textbooks stay as PDFs on disk and the database stores only which lesson is on which page. Copying the prose into the database would create a second version that quietly drifts from the book.                                                                                                           |
| <b>Testing</b>       | pytest + Playwright         | 206 tests against a temporary database, plus browser scripts that click through every screen and fail on any console error.                                                                                                                                                                                   |
| <b>Security</b>      | PBKDF2 (standard library)   | <i>Rejected</i> : bcrypt and argon2 — both need compiling. The stored format keeps its algorithm prefix so we can upgrade later without breaking old rows.                                                                                                                                                    |
| <b>Network</b>       | Portable router             | Neither front end loads a font, a framework or an image from the internet. The pages render identically with the router's upstream unplugged.                                                                                                                                                                 |

#### THE DEPENDENCY LIST, IN FULL

The backend needs six packages: `fastapi`, `uvicorn`, `pydantic`, `pydantic-settings`, `python-dotenv` and `httpx`, plus `pytest` for the tests. That is the entire installation. We specify minimum versions rather than exact pins because exact pins broke on newer Python releases — pip then tried to compile Pydantic from Rust source, which needs a toolchain nobody should have to install to run a study app.

## 4 Following one request through the system

---

This is the densest path in the app and the best one to follow: a child taps an answer in a quiz. Nine steps, four files, and eight database writes — every one of which has to agree with every screen that reads it afterwards.

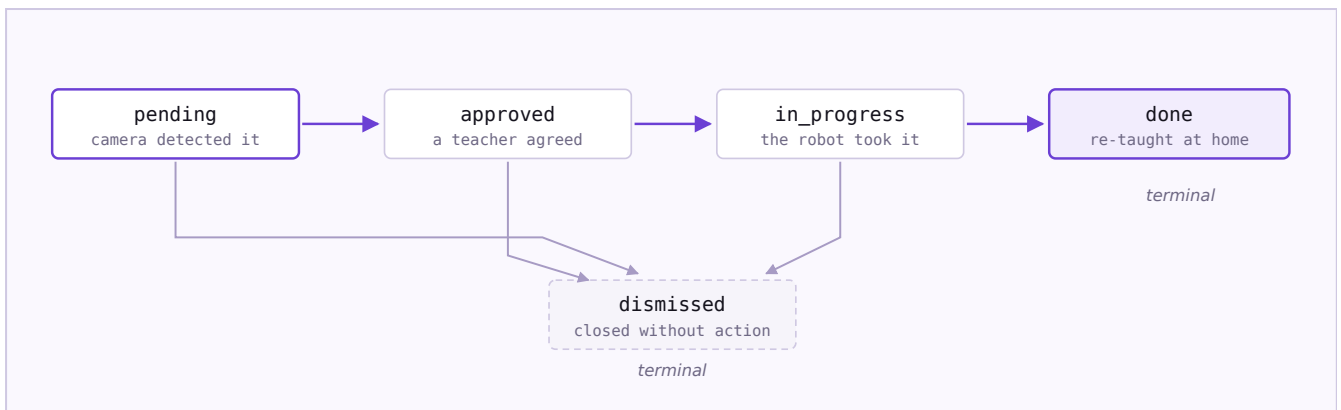
- 1 **The child taps an option.** `app.js` immediately disables all four buttons, so a double-tap cannot submit twice. `static/student/app.js`
- 2 **One request goes out.** `POST /api/students/stu-01/quiz/7/answer` carrying only the index of the option chosen. `static/student/api.js`
- 3 **The server finds the current question** and compares the chosen index with the correct one. The correct index has never been sent to the browser.  
`app/routers/learning.py`
- 4 **The answer is recorded** against the quiz, and the child's streak and remaining lives are updated on the quiz row.
- 5 **Mastery moves.** `economy.update_mastery()` raises the child's mastery of that topic on a correct answer, and lowers it by less than half as much on a wrong one.
- 6 **The reward is granted.** `economy.award()` writes the activity log row, adds stars and XP, applies any streak multiplier, extends the daily streak, re-checks every badge and recalculates the level — all in one place. `app/economy.py`
- 7 **A permanent attempt row is written**, so the child's history survives even if the quiz is later deleted.
- 8 **If that was the last question**, the quiz is marked complete, the completion bonus is awarded and the daily challenge is advanced.
- 9 **One response comes back** describing everything that changed, so the screen knows whether to show a star toast, confetti, or a new badge — without having to ask the server again.

### WHY ONE REQUEST AND NOT SEVEN

The app runs on a tablet sharing a portable router with a camera feed. Every extra round trip is another chance to be visibly waiting while a judge watches. The home screen is built the same way: one call returns the profile, today's lesson, the timetable, the daily challenge and the child's settings together.

### 4.1 The flag lifecycle

The classroom side has its own path, and it is a state machine rather than a sequence. A flag can only move along the arrows below; anything else is refused with an HTTP 409 and nothing is written.



**FIG 2** The flag state machine. Both done and dismissed are terminal — a closed flag can never be reopened. Section 5.2 shows how this diagram exists in the code as a single data structure rather than as scattered conditions.

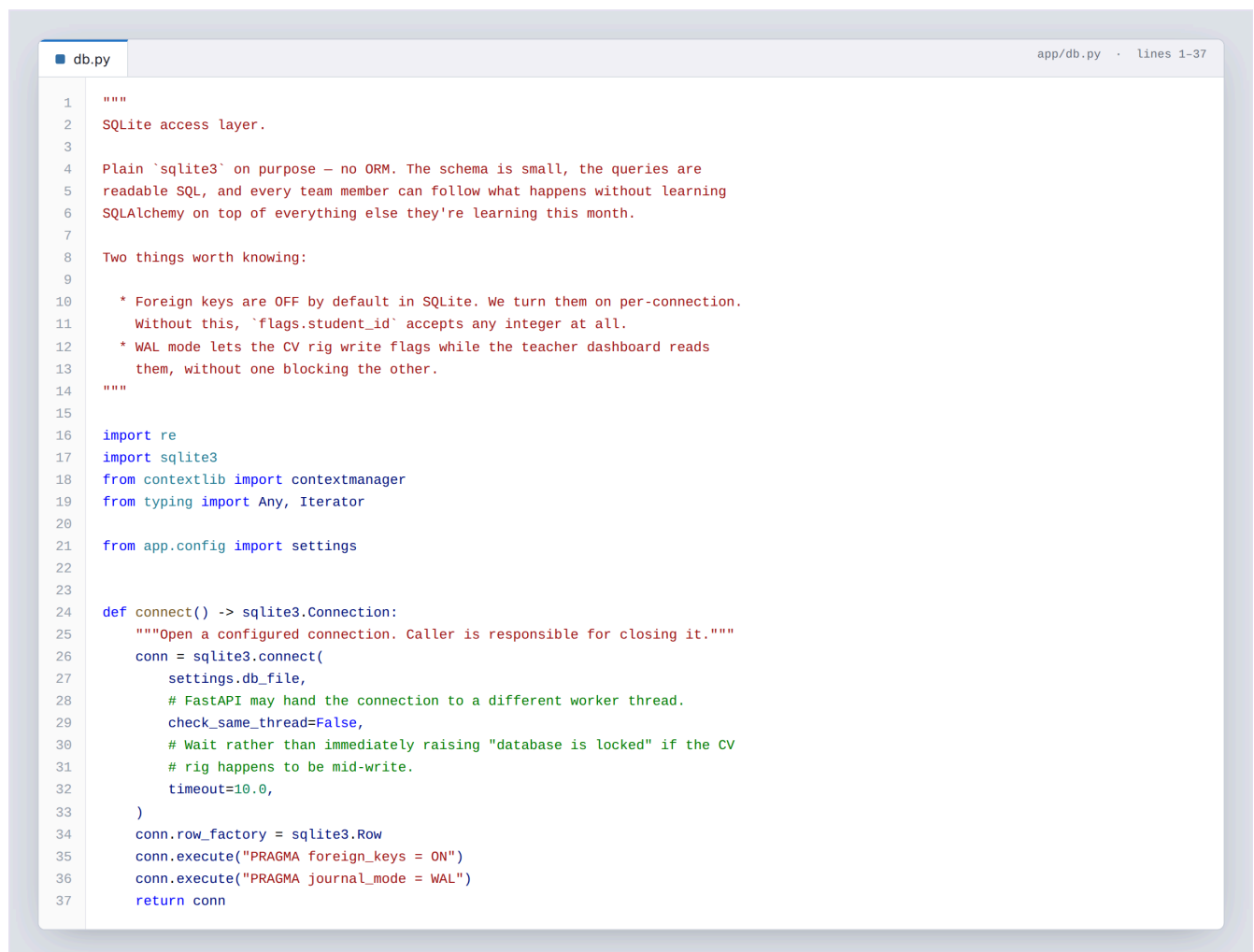
## 5 The code, with commentary

Thirteen extracts, each chosen because it shows a decision rather than routine work. Every one is a screenshot of the real file at the line numbers shown. The complete source is in the repository on the cover.

### 5.1 The database layer

app/db.py · lines 1–37 · whole opening of a 134-line file

Two settings in this file prevent bugs that are very hard to find later. SQLite has foreign keys switched *off* by default, so without line 35 a flag could be stored against a student who does not exist and nothing would complain until a demo. WAL mode on line 36 lets the camera write a flag at the same moment a dashboard is reading one, instead of the two blocking each other.



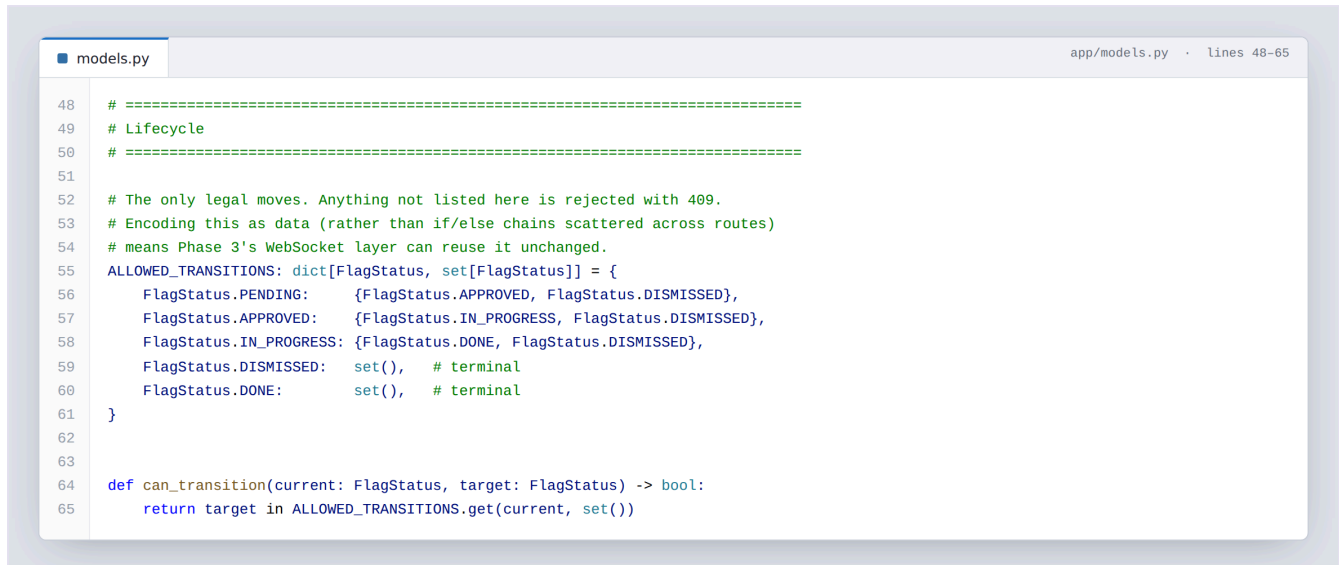
```
1 """
2 SQLite access layer.
3
4 Plain 'sqlite3' on purpose – no ORM. The schema is small, the queries are
5 readable SQL, and every team member can follow what happens without learning
6 SQLAlchemy on top of everything else they're learning this month.
7
8 Two things worth knowing:
9
10 * Foreign keys are OFF by default in SQLite. We turn them on per-connection.
11   Without this, 'flags.student_id' accepts any integer at all.
12 * WAL mode lets the CV rig write flags while the teacher dashboard reads
13   them, without one blocking the other.
14 """
15
16 import re
17 import sqlite3
18 from contextlib import contextmanager
19 from typing import Any, Iterator
20
21 from app.config import settings
22
23
24 def connect() -> sqlite3.Connection:
25     """Open a configured connection. Caller is responsible for closing it."""
26     conn = sqlite3.connect(
27         settings.db_file,
28         # FastAPI may hand the connection to a different worker thread.
29         check_same_thread=False,
30         # Wait rather than immediately raising "database is locked" if the CV
31         # rig happens to be mid-write.
32         timeout=10.0,
33     )
34     conn.row_factory = sqlite3.Row
35     conn.execute("PRAGMA foreign_keys = ON")
36     conn.execute("PRAGMA journal_mode = WAL")
37     return conn
```

**EXTRACT 5.1 app/db.py** — the comment at the top explains *why* there is no ORM, which is the kind of decision a reader six months later cannot recover from the code alone.

## 5.2 The flag lifecycle, written as data

app/models.py · lines 48–65

This is figure 2 turned into code. The obvious way to enforce a lifecycle is a chain of `if` statements inside the route that changes a flag — but then the rule lives in one function, and the next feature that touches flags has to reimplement it. Written as a dictionary, the rule is a value that any part of the system can read, and the live push layer we add next can use it unchanged.



```
models.py app/models.py · lines 48–65
48 # =====
49 # Lifecycle
50 # =====
51
52 # The only legal moves. Anything not listed here is rejected with 409.
53 # Encoding this as data (rather than if/else chains scattered across routes)
54 # means Phase 3's WebSocket layer can reuse it unchanged.
55 ALLOWED_TRANSITIONS: dict[FlagStatus, set[FlagStatus]] = {
56     FlagStatus.PENDING: {FlagStatus.APPROVED, FlagStatus.DISMISSED},
57     FlagStatus.APPROVED: {FlagStatus.IN_PROGRESS, FlagStatus.DISMISSED},
58     FlagStatus.IN_PROGRESS: {FlagStatus.DONE, FlagStatus.DISMISSED},
59     FlagStatus.DISMISSED: set(), # terminal
60     FlagStatus.DONE: set(), # terminal
61 }
62
63
64 def can_transition(current: FlagStatus, target: FlagStatus) -> bool:
65     return target in ALLOWED_TRANSITIONS.get(current, set())
```

**EXTRACT 5.2** `app/models.py` — five states, four legal moves and two terminal states, held as one dictionary. The two empty sets are what make a closed flag impossible to reopen.

## 5.3 Receiving a detection from the classroom

app/routers/flags.py · lines 146–192

A camera is not a reliable narrator. Some detections are confident, some are barely above noise, and a teacher's queue full of false alarms is a queue a teacher stops reading. Rather than filtering inside the vision code — where changing the threshold means editing detection logic at the venue — every detection is stored, and low-confidence ones are stored already closed, with the reason written into the row.

```
146 detected_at = payload.detected_at or utc_now_iso()
147 created_at = utc_now_iso()
148
149 # Low-confidence detections are stored but never queued. Two reasons:
150 # the teacher's queue stays trustworthy, and you keep the filtered rows
151 # as evidence that the noise floor is being handled deliberately.
152 auto_dismissed = (
153     payload.confidence is not None
154     and payload.confidence < settings.flag_min_confidence
155 )
156 initial_status = (
157     FlagStatus.DISMISSED.value if auto_dismissed else FlagStatus.PENDING.value
158 )
159
160 cursor = conn.execute(
161     """
162     INSERT INTO flags (
163         student_id, session_id, source, flag_type,
164         confidence, duration_ms, status,
165         detected_at, created_at, resolution_note, metadata
166     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
167     """,
168     (
169         student["id"],
170         payload.session_id,
171         payload.source.value,
172         payload.flag_type.value,
173         payload.confidence,
174         payload.duration_ms,
175         initial_status,
176         detected_at,
177         created_at,
178         (
179             f"Auto-dismissed: confidence {payload.confidence} below "
180             f"threshold {settings.flag_min_confidence}"
181             if auto_dismissed else None
182         ),
183         json.dumps(payload.metadata) if payload.metadata else None,
184     ),
185 )
186 flag_id = cursor.lastrowid
187
188 _log_event(
189     conn, flag_id, None, initial_status,
190     actor=payload.source.value,
191     note="Flag created" + (" and auto-dismissed" if auto_dismissed else ""),
192 )
```

**EXTRACT 5.3 app/routers/flags.py** — note the SQL: every value is passed as a `?` parameter, never built into the query string. That is the rule that makes SQL injection impossible, and it is followed everywhere in the project without exception.



## 5.4 Refusing an illegal move

app/routers/flags.py · lines 289–323

This is the other half of 5.2. A front end with a bug — or a teammate testing with a REST client — could try to move a flag straight from `pending` to `done`, skipping the teacher entirely. The system refuses, and the error names both the state the flag is in and the moves that *are* allowed from it, so whoever hit the error can fix it without reading the source.

```

289 def transition_flag(
290     flag_id: int,
291     payload: FlagTransition,
292     conn: sqlite3.Connection = Depends(db_dependency),
293 ) -> FlagOut:
294     """
295     pending -> approved | dismissed
296     approved -> in_progress | dismissed
297     in_progress -> done | dismissed
298
299     Illegal moves return 409 rather than silently succeeding, so a buggy
300     frontend can't teleport a flag from pending straight to done.
301     """
302     row = conn.execute("SELECT status FROM flags WHERE id = ?", (flag_id,)).fetchone()
303     if row is None:
304         raise HTTPException(status_code=404, detail=f"Flag {flag_id} not found")
305
306     current = FlagStatus(row["status"])
307     target = payload.status
308
309     if current == target:
310         return _fetch_flag(conn, flag_id)
311
312     if not can_transition(current, target):
313         allowed = sorted(s.value for s in ALLOWED_TRANSITIONS.get(current, set()))
314         raise HTTPException(
315             status_code=409,
316             detail=(
317                 f"Cannot move flag {flag_id} from '{current.value}' to "
318                 f"'{target.value}'. Allowed from '{current.value}': "
319                 f"{allowed or 'nothing (terminal state)'}.",
320             ),
321         )
322
323     now = utc_now_iso()
```

**EXTRACT 5.4** `app/routers/flags.py` — the same request twice in a row returns the flag unchanged rather than failing, because a retry after a dropped connection should not be an error.

## 5.5 One writer for every number

app/economy.py · lines 19–58 and 176–196

A child's star count appears on the home screen, in the weekly chart, on the progress page and in their parents' report. If five different route handlers could each add stars, those four figures would disagree within a week and we would be hunting the difference the night before the competition. So exactly one function is allowed to change a total, and every screen reads from the log that function writes.

The values themselves sit in one block at the top of the file. Two of them are deliberate: a wrong answer is worth 2 stars rather than 0, because for a child who already struggles, zero reward for

effort teaches them to stop trying; and mastery rises by 0.12 on a correct answer but falls by only 0.05 on a wrong one, so a single mistake never wipes out visible progress.

```
economy.py app/economy.py · lines 19-58

19 # =====
20 # Tuning – all reward values in one block, deliberately
21 # =====
22
23 STARS_PER_CORRECT_ANSWER = 10
24 STARS_PER_WRONG_ANSWER = 2 # never zero; effort is worth something
25 STARS_PER_LESSON_STEP = 5
26 STARS_PER_LESSON_COMPLETE = 50
27 STARS_PER_QUIZ_COMPLETE = 30
28 STARS_PER_CHAT_QUESTION = 3
29 STARS_DAILY_CHALLENGE = 100
30
31 XP_PER_CORRECT_ANSWER = 15
32 XP_PER_LESSON_STEP = 8
33 XP_PER_LESSON_COMPLETE = 80
34 XP_PER_QUIZ_COMPLETE = 50
35 XP_PER_GAME_WIN = 40
36 XP_PER_CHAT_QUESTION = 5
37
38 # A quiz streak multiplies stars: 3 in a row = 1.5x, 5 in a row = 2x.
39 STREAK_MULTIPLIERS = {3: 1.5, 5: 2.0, 8: 2.5}
40
41 # XP needed to reach each level. Index = level - 1.
42 # Gentle early curve: a child should reach level 2 in one sitting, because
43 # the first level-up is what teaches them the system rewards them at all.
44 LEVEL_THRESHOLDS = [
45     0, 100, 250, 450, 700, 1000, 1400, 1900, 2500, 3200,
46     4000, 4900, 5900, 7000, 8200, 9500, 11000, 12600, 14300, 16100,
47 ]
48
49 LEVEL_TITLES = [
50     "Beginner", "Learner", "Explorer", "Adventurer", "Thinker",
51     "Problem Solver", "Scholar", "Expert", "Master", "Champion",
52 ]
53
54 # Mastery moves by this much per answer. Correct answers help more than wrong
55 # answers hurt – for a student who already struggles, a single mistake wiping
56 # out visible progress is discouraging in exactly the wrong way.
57 MASTERY_GAIN_CORRECT = 0.12
58 MASTERY_LOSS_WRONG = 0.05
```

**EXTRACT 5.5a app/economy.py** — every tunable value in the game economy, in one screen. Balancing the rewards means editing here and nowhere else.

```
economy.py app/economy.py · lines 176-196

176 def award(
177     conn: sqlite3.Connection,
178     student_id: int,
179     activity_type: str,
180     *,
181     stars: int = 0,
182     xp: int = 0,
183     subject_id: int | None = None,
184     topic_id: int | None = None,
185     reference_id: int | None = None,
186     duration_s: int = 0,
187     detail: str | None = None,
188     multiplier: float = 1.0,
189 ) -> AwardResult:
190     """
191     Grant stars and XP, log the activity, update the streak, check for badges
192     and level-ups. The only function permitted to change a student's totals.
193
194     Caller supplies an open transaction; this does not commit.
195     """
196     stars_delta = int(round(stars * multiplier))
```

**EXTRACT 5.5b app/economy.py** — the docstring states the rule the whole design depends on. It does not commit; the caller owns the transaction, so a failure part-way through leaves no half-granted reward.

## 5.6 Finding the right page of the book

app/services/rag.py · lines 163–216

Before Souly explains anything, it has to find which page of the real textbook the child's question is about. This is our own scoring function: how often a word appears on a page, weighted down if it appears on every page, plus a bonus when the word matches the lesson title — because a child's question is usually a topic name rather than a sentence. A page we cannot read is skipped outright at lines 169 to 172 — the comment there is the rule: retrieval must not invent a page it cannot read.

```
rag.py app/services/rag.py · lines 163–216

163 # Document frequency across the candidate set, so a word appearing on
164 # every page ("lesson", "number") counts for less than a rare one.
165 docs = []
166 df: dict[str, int] = {}
167 for row in rows:
168     body = curriculum.page_text(row["book_code"], row["page"])
169     if not body:
170         # Not ingested, or OCR produced nothing. Retrieval must not invent
171         # a page it cannot read.
172         continue
173     stems = _stems(f"{row['lesson']} {row['topic_title'] or ''} {body}")
174     docs.append((row, body, stems, set(stems)))
175     for stem in set(stems):
176         df[stem] = df.get(stem, 0) + 1
177
178 if not docs:
179     return []
180
181 n_docs = len(docs)
182 scored: list[Chunk] = []
183
184 for row, body, stems, stem_set in docs:
185     score = 0.0
186     for q in query_stems:
187         if q not in stem_set:
188             continue
189         tf = stems.count(q) / len(stems)
190         idf = math.log((n_docs + 1) / (df.get(q, 0) + 1)) + 1.0
191         score += tf * idf
192
193     # Lesson-title matches are a strong signal for a child's question,
194     # which is usually a topic name ("help with fractions") rather than a
195     # sentence.
196     title_stems = set(_stems(f"{row['lesson']} {row['topic_title'] or ''}"))
197     score += len(set(query_stems) & title_stems) * 0.35
198
199 if score <= 0:
200     continue
201
202 scored.append(Chunk(
203     text=body,
204     score=score,
205     page_id=row["page_id"],
206     page=row["page"],
207     lesson=row["lesson"],
208     book_id=row["book_id"],
209     book_title=row["book_title"],
210     subject_name=row["subject_name"] or "",
211     topic_id=row["topic_id"],
212     topic_title=row["topic_title"],
213 ))
214
215 scored.sort(key=lambda c: c.score, reverse=True)
216 return scored[:limit]
```

**EXTRACT 5.6** `app/services/rag.py` — the search only ever looks at books a human has marked as verified, so the tutor physically cannot teach from material nobody checked.

## 5.7 Canon, rendition and path

app/services/tutor.py · lines 14–44 · from a 1,967-line file

This comment is the central idea of the whole project, so it is written down where the code that implements it lives. The textbook page is the *canon* and nothing may change it. The explanation is the *rendition*, written for one particular child at the moment they open the page. The order of lessons is the *path*, and it is decided by plain rules over their mastery, not by the model.

The reason the canon is frozen matters more than it sounds: a wrong step invented inside a mathematics procedure is invisible to a child who is already struggling, because they assume the mistake is theirs.

```
tutor.py app/services/tutor.py · lines 14–44
14     answer()          free-form question, grounded, still hint-shaped
15     should_offer_help() has this student stalled?
16
17     -----
18     CANON, RENDITION, PATH
19     -----
20     Since schema_v5 the content is a Ministry PDF and nothing here paraphrases it
21     into the database. Three layers:
22
23         canon      the PDF page.      Nobody writes it. It stays on disk.
24         rendition  the explanation.    This module writes it, per child, from the
25                                         page – cached in `page_renditions`.
26         path       order and pacing.   A deterministic policy over mastery.
27
28     The canon is frozen because a hallucinated step in a maths procedure is
29     invisible to a child who is already struggling: they assume they are the one
30     who is wrong. Every function below that talks to the model is handed the
31     page's text as grounding and told it may not go beyond it. When there is no
32     text for a page – never ingested, or the OCR came back empty – the answer is
33     to say so, not to let the model fill the silence.
34
35     -----
36     WHY HELP IS OFFERED, NOT WAITED FOR
37     -----
38     Grainger, Williams & Lind (2016) found autistic children's confidence
39     judgements are significantly less accurate, and that they "used monitoring to
40     influence control processes significantly less than neurotypical children."
41
42     Plainly: the link between not understanding and doing something about it is
43     weaker. A button saying "tap if you're stuck" gets pressed by the students who
44     need it least.
```

**EXTRACT 5.7 app/services/tutor.py** — two children on the same page get the same facts in different words. That is the design, stated in the file that implements it.

## 5.8 Why Souly never gives the answer

app/services/llm.py · lines 19–49

This is the single design rule we would most want a judge to ask us about. A study tool that answers questions makes practice look better and learning worse — and there is published evidence for that, which is why the finding is recorded in the file that enforces the rule rather than only in a slide. Souly is given the worked solution so it can steer, and is forbidden from repeating it.

```
llm.py app/services/llm.py · lines 19–49
19 -----
20 THE HINT RULE
21 -----
22 Souly does not give answers. It gives hints.
23
24 This is not a style preference. Bastani et al. (PNAS 2025) gave ~1,000
25 students access to an unrestricted GPT tutor: their practice performance rose
26 48%, and on a later exam WITHOUT the tutor they scored 17% BELOW students who
27 never had it. A guardrailed tutor – hints only, with the worked solution in
28 its prompt – produced +127% in practice and no exam penalty.
29
30 67% of student messages to the unrestricted tutor were requests for the answer.
31
32 So: SOLUTION_PROMPT_RULES below is load-bearing, and the worked solution and
33 common wrong answers for the specific item are passed into every hint call.
34 -----
35 """
36
37 import base64
38 import json
39 import time
40 import re
41 from dataclasses import dataclass, field
42 from typing import Any
43
44 import httpx
45
46 from app.config import settings
47
48 GEMINI_BASE = "https://generativelanguage.googleapis.com/v1beta"
49
```

**EXTRACT 5.8** `app/services/llm.py` — a comment that explains a constraint by citing why it exists. Anyone tempted to relax it later reads the reason first.

## 5.9 What happens when the network fails

app/services/llm.py · lines 634–668

At a competition venue the network will fail at least once. A tutoring app that returns an error because an upstream service hiccupped is worse than one that answers more plainly, so every call has a defined fallback: the retrieved textbook page itself, delivered in Souly's voice. The reply is tagged with which engine produced it, so the screen can show an honest indicator and we always know which path ran.

```
634 def _fallback(prompt: str, context: str, *, reason: str, latency_ms: int = 0,
635               suggestions: list[str] | None = None,
636               fallback_text: str | None = None) -> LLMResponse:
637     """
638     Answer without an LLM.
639
640     Prefers a caller-supplied line (a stored hint, the lesson text). Otherwise
641     reads the retrieved material and returns the most relevant sentences in
642     Souly's voice. Grounded, correct, always available.
643     """
644     # Deterministic pick so the same question gives the same greeting – young
645     # users with autism in particular do better with predictable responses.
646     opener = _ENCOURAGEMENTS[len(prompt) % len(_ENCOURAGEMENTS)]
647
648     if fallback_text:
649         return LLMResponse(text=fallback_text, engine="fallback",
650                           latency_ms=latency_ms, error=reason)
651
652     if not context.strip():
653         if suggestions:
654             examples = suggestions[:2]
655             joined = " or ".join(examples) if len(examples) > 1 else examples[0]
656             tail = f"Try asking me about {joined}, and I'll explain it."
657         else:
658             tail = ("I don't have any lessons loaded yet. Ask your teacher to add "
659                   "some, and then I can help you with them.")
660         return LLMResponse(text=f"{opener} I haven't learned about that one yet. {tail}",
661                           engine="fallback", latency_ms=latency_ms, error=reason)
662
663     body_lines = [line for line in context.splitlines()
664                  if line.strip() and not re.match(r"^\s*\d+\s$", line.strip())]
665     body = " ".join(body_lines)
666     answer = _first_sentences(body, 3)
667     if len(answer) > 320:
668         answer = _first_sentences(body, 2)
```

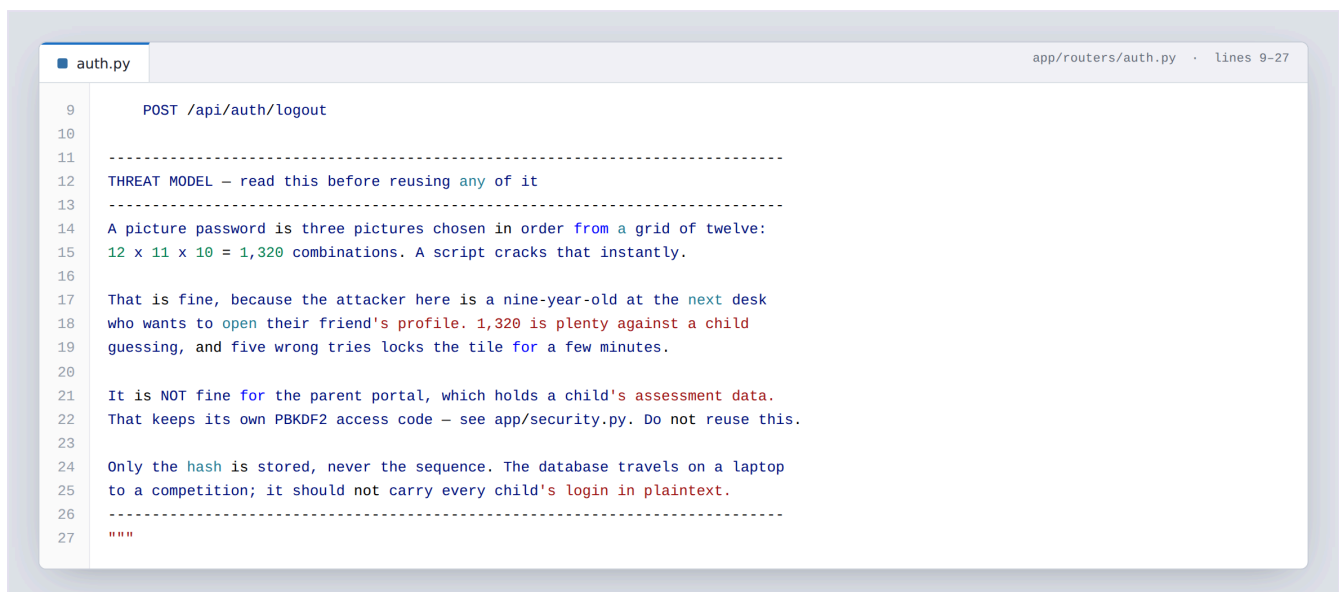
**EXTRACT 5.9** `app/services/llm.py` — the failure path is designed, not accidental. It is chosen in advance instead of improvised on stage.

## 5.10 Security sized to the user

app/routers/auth.py · lines 9–27

A child signs in by tapping three pictures from a grid of twelve. That is 1,320 combinations — trivial for a computer to guess, and we say so in the file. It is the right choice anyway, because the attacker we are actually defending against is the classmate at the next desk, a nine-year-old cannot type a password ten times a day, and five wrong tries lock the tile for three minutes.

The parents' hub is a different problem and gets a different answer. It holds a child's assessment data, so it uses a long random access code hashed with PBKDF2, stores no plaintext, and shows no list of parent names — publishing a school's family list to anyone who loads the page would be a worse leak than a weak password. The two systems also use separate token tables, so a student's token cannot satisfy a parent's check no matter how a query is written. There is a test for exactly that.



```
9      POST /api/auth/logout
10
11      -----
12      THREAT MODEL – read this before reusing any of it
13      -----
14      A picture password is three pictures chosen in order from a grid of twelve:
15      12 x 11 x 10 = 1,320 combinations. A script cracks that instantly.
16
17      That is fine, because the attacker here is a nine-year-old at the next desk
18      who wants to open their friend's profile. 1,320 is plenty against a child
19      guessing, and five wrong tries locks the tile for a few minutes.
20
21      It is NOT fine for the parent portal, which holds a child's assessment data.
22      That keeps its own PBKDF2 access code – see app/security.py. Do not reuse this.
23
24      Only the hash is stored, never the sequence. The database travels on a laptop
25      to a competition; it should not carry every child's login in plaintext.
26      -----
27      """
```

**EXTRACT 5.10 app/routers/auth.py** — the comment names the threat the design defends against, and names the case where reusing it would be wrong.

## 5.11 The front end holds no truth

static/student/api.js · lines 54–80

Every network call in the student app goes through this one function, so retries, timeouts, the offline indicator and the session token are handled once instead of in ninety places. The small rewrite on lines 56 to 58 lets every screen ask for `/api/me/...` while the server only ever sees a real student id, which means switching which child is signed in changes one variable and no screens.



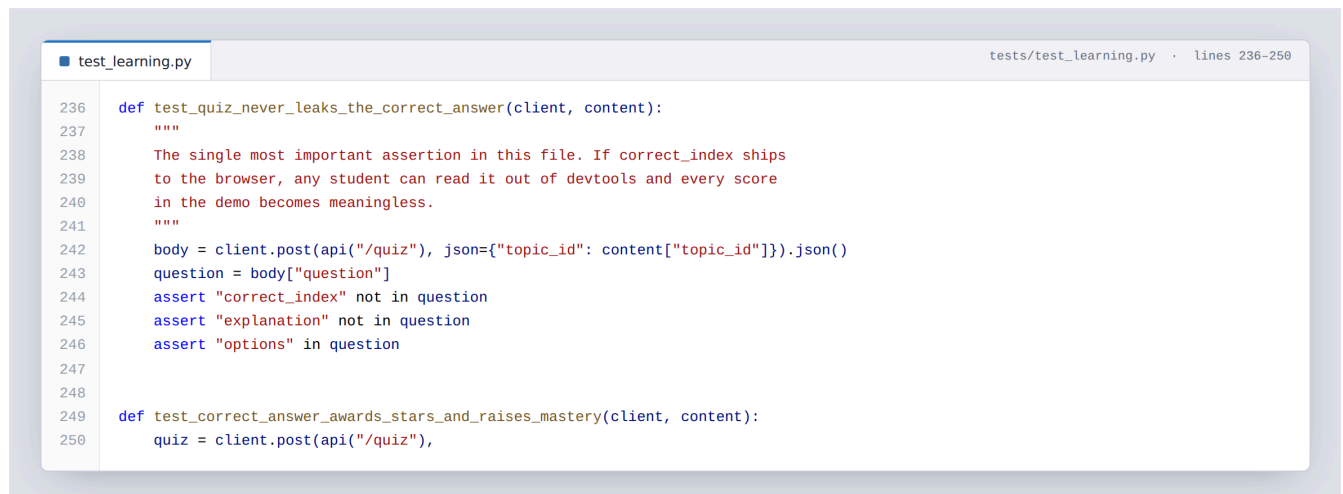
```
54 }
55
56 function url(path) {
57   if (path.startsWith('/api/me')) {
58     return BASE + path.replace('/api/me', `/api/students/${studentId}`);
59   }
60   return BASE + path;
61 }
62
63 async function request(path, options = {}, attempt = 0) {
64   const controller = new AbortController();
65   const timer = setTimeout(() => controller.abort(), options.timeout || TIMEOUT_MS);
66
67   try {
68     const response = await fetch(url(path), {
69       ...options,
70       signal: controller.signal,
71       headers: {
72         ...(options.body instanceof FormData
73           ? {}
74           : { 'Content-Type': 'application/json' }),
75         ...(token ? { 'X-Souly-Token': token } : {}),
76         ...(options.headers || {})
77       }
78     });
79     clearTimeout(timer);
80     setOnline(true);
```

**EXTRACT 5.11 static/student/api.js** — none of the ten screens holds a hardcoded number. A figure that cannot disagree with the database is a figure that cannot be wrong in front of a judge.

## 5.12 The test that protects the demo

tests/test\_learning.py · lines 236–250

A quiz question travels to the browser as JSON. If the correct answer travelled with it, any child could open the developer tools and read it — and every score in our demonstration would be meaningless. So the correct index is never sent until after the answer is submitted, and this test fails the build if that ever changes.



```
236 def test_quiz_never_leaks_the_correct_answer(client, content):
237     """
238     The single most important assertion in this file. If correct_index ships
239     to the browser, any student can read it out of devtools and every score
240     in the demo becomes meaningless.
241     """
242     body = client.post(api("/quiz"), json={"topic_id": content["topic_id"]}).json()
243     question = body["question"]
244     assert "correct_index" not in question
245     assert "explanation" not in question
246     assert "options" in question
247
248
249 def test_correct_answer_awards_stars_and_raises_mastery(client, content):
250     quiz = client.post(api("/quiz"),
```

**EXTRACT 5.12 tests/test\_learning.py** — the test's name is a sentence describing the guarantee. A failing test then reads as a statement about what broke.

## 5.13 The whole suite, running

206 tests · 5.0 seconds · clean checkout

Tests are only worth writing if they are fast enough to run constantly. The whole suite finishes in five seconds against a temporary database, and a guard aborts the run if it ever touches the real one — because the first version of our fixtures silently wiped a development database, and we would rather that fail loudly than quietly.

```
souly-backend - pytest

$ pytest -v

tests/test_spine.py::test_unknown_student_is_rejected PASSED [ 90%]
tests/test_spine.py::test_inactive_student_is_rejected PASSED [ 91%]
tests/test_spine.py::test_invalid_flag_type_is_rejected PASSED [ 91%]
tests/test_spine.py::test_out_of_range_confidence_is_rejected PASSED [ 92%]
tests/test_spine.py::test_malformed_detected_at_is_rejected PASSED [ 92%]
tests/test_spine.py::test_low_confidence_flag_is_stored_but_not_queued PASSED [ 93%]
tests/test_spine.py::test_full_lifecycle_pending_to_done PASSED [ 93%]
tests/test_spine.py::test_illegal_transition_is_refused PASSED [ 94%]
tests/test_spine.py::test_terminal_state_cannot_be_reopened PASSED [ 94%]
tests/test_spine.py::test_transition_to_same_status_is_idempotent PASSED [ 95%]
tests/test_spine.py::test_transition_on_missing_flag_is_404 PASSED [ 95%]
tests/test_spine.py::test_every_transition_is_recorded PASSED [ 96%]
tests/test_spine.py::test_refused_transition_leaves_no_audit_entry PASSED [ 96%]
tests/test_spine.py::test_pending_queue_is_newest_first PASSED [ 97%]
tests/test_spine.py::test_pending_filter_by_student_excludes_others PASSED [ 97%]
tests/test_spine.py::test_list_flags_by_status PASSED [ 98%]
tests/test_spine.py::test_get_missing_flag_is_404 PASSED [ 98%]
tests/test_spine.py::test_health_reports_integrations_as_unconfigured PASSED [ 99%]
tests/test_spine.py::test_api_root_endpoint PASSED [ 99%]
tests/test_spine.py::test_root_serves_the_student_app PASSED [100%]

===== 206 passed in 5.00s =====
```

**EXTRACT 5.13** The end of a full run. The test names are readable as sentences: a low-confidence flag is stored but not queued, an illegal transition is refused, a terminal state cannot be reopened.

## 6 Database design

---

One SQLite file holds 41 tables and 9 views. They are built by seven migration files applied in order, every one of them safe to run again — which is why nobody on the team can break their database by running setup twice.

### 6.1 The five groups

| GROUP             | TABLES                                                                                                                          | WHAT IT ANSWERS                                                    |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <b>People</b>     | students, teachers, parents, parent_student, auth_tokens, parent_tokens                                                         | Who is who, and which parent may see which child.                  |
| <b>Curriculum</b> | subjects, topics, curriculum_books, curriculum_pages, page_renditions, questions                                                | What gets taught, and which page of which book it is on.           |
| <b>Activity</b>   | sessions, attempts, quizzes, quiz_questions, lesson_progress, page_activity, game_plays, chat_messages, hint_requests           | What the child actually did.                                       |
| <b>Progress</b>   | mastery, activity_log, student_skills, badges, student_badges, rewards, student_rewards, weekly_goals, daily_challenge_progress | How they are doing, and what they have earned.                     |
| <b>Classroom</b>  | flags, flag_events                                                                                                              | What the camera noticed, and everything that happened to it since. |

Plus student\_settings, learner\_profiles, onboarding\_responses, teacher\_notes, conversations and schedule\_items.

## 6.2 Seven migrations, in order

| FILE          | LINES | WHAT IT ADDS                                                                                        |
|---------------|-------|-----------------------------------------------------------------------------------------------------|
| schema.sql    | 390   | People, sessions, and the flag spine.                                                               |
| schema_v2.sql | 492   | Everything the student screens need — quizzes, badges, rewards, the activity log.                   |
| schema_v3.sql | 341   | The data the hint ladder needs, and a record of every help request so we can show the ladder works. |
| schema_v4.sql | 264   | Sign-in: avatars, picture passwords, tokens, and the learner profile.                               |
| schema_v5.sql | 492   | Replaces our written lesson text with a map into the real Ministry books.                           |
| schema_v6.sql | 37    | Makes a cached explanation expire when what we know about the child changes.                        |
| schema_v7.sql | 168   | The parents' hub: its own tokens, teacher notes, and per-child message threads.                     |

## 6.3 Two habits worth pointing at

**Bad data fails loudly.** Columns that can only hold a few values carry a `CHECK` constraint, so a typo becomes an error at the moment of writing rather than a fifth flag status nobody notices for a month. Timestamps are stored as ISO-8601 text, which sorts correctly as text and reads correctly to a human.

**Access control lives in the schema, not in the routes.** A parent's link to a child is a row in `parent_student`, and every parent-facing query joins through it. If the join returns nothing, the answer is 404. The alternative — trusting a child id that arrived from a browser — is one forgotten check away from showing a parent someone else's child.

### WHY MIGRATIONS RATHER THAN ONE FILE

The schema changed six times as the design settled, and each change had to work against databases that already had data in them. Keeping the history as separate files means the reason for each change is recorded next to the change, and a teammate whose database is three versions old catches up by running setup again.

## 7 How we write and check our code

---

### 7.1 Comments explain decisions, not syntax

A comment that says `# loop through the students` above a loop through the students is noise. The comments in this project answer the question a reader cannot answer from the code: *why is it like this?* Every module opens with a short statement of what it is for and what was decided; inline comments appear where a line would otherwise look arbitrary or wrong.

Extracts 5.7, 5.8 and 5.10 are all comments rather than logic, and they are three of the most valuable things in the repository — they are the only record of arguments the team had and settled.

### 7.2 The rules we hold to

| RULE                              | WHAT IT MEANS IN PRACTICE                                                                                                                          |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>One writer per fact</b>        | Only <code>economy.py</code> changes a score. Only <code>config.py</code> reads the environment. Only the ingest script writes the curriculum map. |
| <b>No magic numbers</b>           | Every threshold and reward is a named constant in one block, not a bare number buried in a function.                                               |
| <b>Parameterised SQL, always</b>  | Values are passed as <code>?</code> placeholders. No query is ever built by joining strings together.                                              |
| <b>Fail with a useful message</b> | An error says what was wrong and what would have been valid — see extract 5.4.                                                                     |
| <b>The browser holds no truth</b> | Screens draw from live API data. Refreshing the page loses nothing, because nothing was only in the browser.                                       |
| <b>Names are sentences</b>        | <code>test_low_confidence_flag_is_stored_but_not_queued</code> needs no comment to explain it.                                                     |
| <b>Every layer degrades</b>       | No key, no network, no OCR text — each has a defined behaviour that is not a crash.                                                                |

### 7.3 What the tests cover

| FILE                   | TESTS | AREA                                                                                                                                                        |
|------------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tests/test_learning.py | 138   | Lessons, pages, practice generation, quizzes, hints, the economy, the tutor.                                                                                |
| tests/test_parent.py   | 35    | The parents' hub, including that a parent can only ever reach their own children.                                                                           |
| tests/test_spine.py    | 24    | The flag lifecycle: validation, the noise floor, every legal and illegal transition, the audit trail.                                                       |
| tests/test_schema.py   | 9     | The database builds correctly and its constraints actually reject bad data.                                                                                 |
| tests/ui_smoke.py      | –     | Drives all ten screens in a real browser, buys a reward, checks the stars really left the balance, screenshots everything. Any console error fails the run. |
| tests/gate_smoke.py    | –     | Drives sign-in and the entry activity from a cold start, the way a child would meet it.                                                                     |

206 automated tests in total, running in five seconds against a temporary database.

#### THE CHECK THAT MATTERS MOST

Two tests exist purely to protect promises we make out loud:

`test_quiz_never_leaks_the_correct_answer` and `test_parent_sees_only_their_own_child`. If either ever fails, a claim in our presentation has stopped being true — and we find out in five seconds rather than on stage.

## 8 What runs today, and what is still ahead

Everything in the left column can be demonstrated on our own hardware. Everything in the right column is designed and partly built, and we would rather say so here than have it discovered.

| CAPABILITY               | STATE    | DETAIL                                                                                                                                                                                   |
|--------------------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Flag pipeline</b>     | WORKING  | Receive, filter by confidence, queue, approve, claim, resolve, with every transition recorded. 24 tests.                                                                                 |
| <b>Real curriculum</b>   | WORKING  | Two Ministry books, 14 lessons, 106 mapped pages, cited by book and page number.                                                                                                         |
| <b>Student app</b>       | WORKING  | Ten screens, all drawn from live data: sign-in, entry activity, lessons, quizzes, games, rewards, badges, progress, profile.                                                             |
| <b>Parents' hub</b>      | WORKING  | Eight screens, multiple children per code, teacher notes and per-child message threads.                                                                                                  |
| <b>Tutor and hints</b>   | WORKING  | Explanations written per child from the real page, four-rung hint ladder, full voice loop.                                                                                               |
| <b>Offline behaviour</b> | WORKING  | Every external service has a defined fallback. The app runs with no keys at all.                                                                                                         |
| <b>Classroom vision</b>  | SEPARATE | Owned by another team, in another repository. Our side of the contract is finished, documented in <a href="#">docs/CV_INTEGRATION.md</a> , and testable today with a simulator we wrote. |
| <b>Teacher dashboard</b> | AHEAD    | The API it needs exists and is tested. The screen itself is not built yet.                                                                                                               |
| <b>Live push</b>         | AHEAD    | Flags are read by polling today. The state machine in extract 5.2 was written so a live push layer can reuse it without changes.                                                         |
| <b>Speech vendor</b>     | OPEN     | Three providers implemented; which one we ship is one environment variable and is still being decided on cost and offline behaviour.                                                     |



## 8.1 Two constraints we chose to keep

**Souly reports engagement, never emotion.** The classroom system measures where a child is looking and for how long. It does not claim to detect how they feel. Inferring emotion from a face is scientifically contested, and a system that tells a teacher a child is "sad" is making a claim it cannot support about a child it is supposed to protect.

**A teacher approves before anything reaches a classroom.** The system never acts on a flag by itself. A detection is a suggestion to an adult, and no screen visible to a whole class ever shows an individual child's name next to a difficulty.

### WHY WE ARE STATING THE GAPS

Every one of these is a decision about where six weeks of eight students' time went, and we would rather explain that reasoning than have the list read as an oversight. The pieces that are finished are finished properly — tested, documented and demonstrable — which we think is worth more than a wider surface that only works when nothing goes wrong.

## 9 Running the system yourself

---

Everything below works from a clean checkout on Windows, macOS or Linux. No database server, no build step, no account required — the app runs without any API keys, using its offline paths.

- 1 **Get the code and create an environment.**  
`git clone https://github.com/beshoywael-him/souly-backend`  
`python -m venv .venv` then activate it  
`pip install -r requirements.txt` — six packages.
- 2 **Build the database.** `python scripts/init_db.py`  
 Creates 41 tables and 9 views from the seven migration files.
- 3 **Add the people and the scaffolding.**  
`python scripts/seed_students.py`  
`python scripts/seed_content.py`  
`python scripts/seed_parents.py` — prints the parent access codes.
- 4 **Load the textbooks.**  
`python scripts/render_pages.py` then `python scripts/ingest_curriculum.py`  
 Reads the two Ministry PDFs and the approved lesson map. Add `--dry-run` to see what it would do without writing anything.
- 5 **Start the server.** `run.bat` on Windows, `./run.sh` elsewhere.  
 It binds to all interfaces, so every device on the router reaches it.
- 6 **Open the student app** at `localhost:8000/student`  
 Choose a face, tap three pictures, and confirm them. The entry activity runs once.
- 7 **Open the parents' hub** at `localhost:8000/parent`  
 Sign in with any access code printed in step 3.
- 8 **Inspect the system.** `localhost:8000/docs` is the live API contract — all 83 operations, generated from the code, with a working test button on each.  
`localhost:8000/health` reports which services have keys and how much curriculum is loaded.
- 9 **Watch the classroom pipeline** without any camera hardware:  
`python scripts/fake_cv_publisher.py --watch --interval 3`  
`python scripts/poll_pending.py --watch`
- 10 **Run the tests.** `pytest` — 206 tests, about five seconds.  
`python tests/ui_smoke.py` drives every screen in a real browser.

#### IF YOU ONLY HAVE FIVE MINUTES

Steps 1, 2, 3 and 5, then open /student. You will reach the sign-in screen and the whole app; the lessons need step 4 because the content is the textbooks themselves and they are read from disk, not stored in the database.

## 10 Complete file inventory

Every file in the repository, excluding generated caches and the two textbook PDFs. This is the whole of what is at the address on the cover.

### Backend — app/

| FILE                        | LINES         |
|-----------------------------|---------------|
| app/__init__.py             | 3             |
| app/config.py               | 125           |
| app/db.py                   | 134           |
| app/deps.py                 | 47            |
| app/economy.py              | 557           |
| app/main.py                 | 118           |
| app/models.py               | 215           |
| app/onboarding_items.py     | 323           |
| app/routers/__init__.py     | 0             |
| app/routers/auth.py         | 357           |
| app/routers/flags.py        | 375           |
| app/routers/gamification.py | 423           |
| app/routers/health.py       | 49            |
| app/routers/learning.py     | 1,217         |
| app/routers/onboarding.py   | 498           |
| app/routers/parent.py       | 965           |
| app/routers/parent_auth.py  | 303           |
| app/routers/progress.py     | 179           |
| app/routers/student.py      | 316           |
| app/routers/tutor_api.py    | 426           |
| app/security.py             | 58            |
| app/services/__init__.py    | 1             |
| app/services/curriculum.py  | 437           |
| app/services/llm.py         | 886           |
| app/services/rag.py         | 250           |
| app/services/stt.py         | 147           |
| app/services/tts.py         | 251           |
| app/services/tutor.py       | 1,967         |
| <b>subtotal</b>             | <b>10,627</b> |

### Front ends — static/

| FILE                      | LINES        |
|---------------------------|--------------|
| static/parent/api.js      | 246          |
| static/parent/app.js      | 447          |
| static/parent/base.css    | 908          |
| static/parent/index.html  | 49           |
| static/parent/pages.js    | 1,203        |
| static/student/api.js     | 361          |
| static/student/app.css    | 1,462        |
| static/student/app.js     | 1,173        |
| static/student/base.css   | 516          |
| static/student/games.js   | 270          |
| static/student/gate.js    | 620          |
| static/student/index.html | 113          |
| static/student/pages.js   | 936          |
| <b>subtotal</b>           | <b>8,304</b> |

## Operations — scripts/

| FILE                         | LINES        |
|------------------------------|--------------|
| scripts/__init__.py          | 0            |
| scripts/check_image.py       | 106          |
| scripts/check_keys.py        | 104          |
| scripts/clear_curriculum.py  | 129          |
| scripts/demo_spine.py        | 173          |
| scripts/fake_cv_publisher.py | 172          |
| scripts/generate_practice.py | 193          |
| scripts/ingest_curriculum.py | 559          |
| scripts/init_db.py           | 65           |
| scripts/make_cert.py         | 287          |
| scripts/poll_pending.py      | 94           |
| scripts/render_pages.py      | 324          |
| scripts/seed_content.py      | 318          |
| scripts/seed_demo_lesson.py  | 29           |
| scripts/seed_parents.py      | 299          |
| scripts/seed_students.py     | 326          |
| <b>subtotal</b>              | <b>3,178</b> |

## Tests — tests/

| FILE                   | LINES        |
|------------------------|--------------|
| tests/__init__.py      | 0            |
| tests/conftest.py      | 118          |
| tests/gate_smoke.py    | 272          |
| tests/test_learning.py | 2,169        |
| tests/test_parent.py   | 452          |
| tests/test_schema.py   | 163          |
| tests/test_spine.py    | 328          |
| tests/ui_smoke.py      | 261          |
| <b>subtotal</b>        | <b>3,763</b> |

## Documentation — docs/

| FILE                           | LINES        |
|--------------------------------|--------------|
| docs/ARCHITECTURE.md           | 433          |
| docs/CURRICULUM.md             | 206          |
| docs/CV_INTEGRATION.md         | 172          |
| docs/PERSONALISATION-DESIGN.md | 468          |
| docs/RESEARCH-EVIDENCE.md      | 458          |
| docs/STACK.md                  | 86           |
| <b>subtotal</b>                | <b>1,823</b> |

## Curriculum — data/

| FILE                                    | LINES      |
|-----------------------------------------|------------|
| data/curriculum/5th primary math.pdf    | —          |
| data/curriculum/5th primary science.pdf | —          |
| data/curriculum/curriculum_map.json     | 134        |
| <b>subtotal</b>                         | <b>134</b> |

## Project root

| FILE             | LINES        |
|------------------|--------------|
| README.md        | 333          |
| pytest.ini       | 6            |
| requirements.txt | 28           |
| run-https.bat    | 46           |
| run.bat          | 26           |
| run.sh           | 11           |
| schema.sql       | 390          |
| schema_v2.sql    | 492          |
| schema_v3.sql    | 341          |
| schema_v4.sql    | 264          |
| schema_v5.sql    | 492          |
| schema_v6.sql    | 37           |
| schema_v7.sql    | 168          |
| <b>subtotal</b>  | <b>2,634</b> |